

ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

3er curso

TEMA 1. Arquitectura de un multiprocesador- en-un-chip

Departamento de Ingeniería y Tecnología de Computadores
Universidad de Murcia

Agenda

- 1.1 Aspectos tecnológicos: Leyes de Moore y de Dennard
- 1.2 Clasificación de Flynn. Principales figuras de mérito
- 1.3 Aspectos paralelos: Leyes de Amdahl y Gustafson
- 1.4 De sistemas empujados a supercomputadores

Arquitectura de computadores

- La arquitectura de un computador está compuesta por:
 - ISA (Instruction Set Architecture)
 - Conjunto de instrucciones que pueden ser ejecutadas por el procesador.
 - Ejemplos: x86, ARM, RISC-V
 - Microarquitectura
 - Cómo se implementan las instrucciones del ISA
 - Diseño tecnológico del sistema $\equiv$ hardware

Evolución de las microarquitecturas (resumen)

- Monociclo: Simplicidad, pero gran ineficiencia
- Multiciclo: ¿cómo mejora al monociclo?
- Segmentación
 - Ejecución **solapada** de varias instrucciones
 - Detenciones debido a los riesgos y **dependencias**
- **Explotando el ILP**
 - Superescalares, Superpipelining,
 - MultiThreading, Vectorización, ...
- **CMPs (multiprocesadores en un chip)**
- Arquitecturas heterogéneas
 - CMPs heterogéneos
 - CMP + aceleradores (GPUs, DSPs, TPUs, XPU)
- Supercomputadores

Transistores y Cables

- Tamaño característico (*Feature size*)
 - Tamaño mínimo del transistor en la dimensión X o Y
 - **10** micras en 1971 y **10** nanómetros en 2016
 - El rendimiento del transistor escala linealmente
 - El retardo en los cables no mejora con el tamaño característico
 - La densidad de integración escala cuadráticamente

Tendencias en tecnología (I)

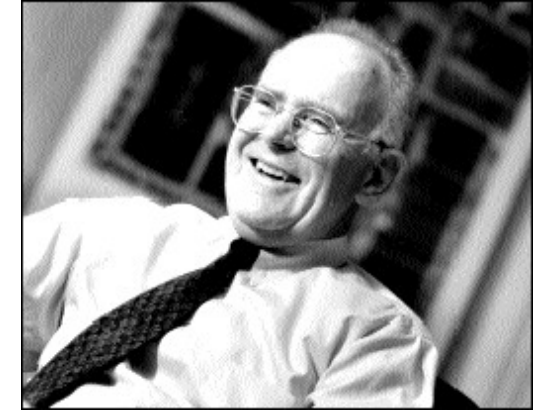
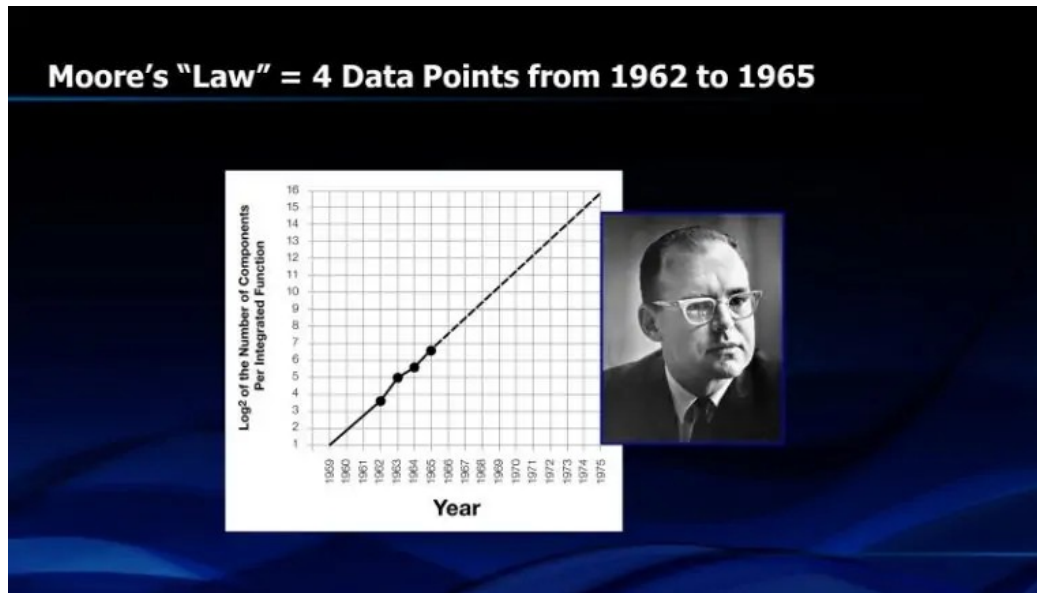
- Circuitos Integrados (**Ley de Moore**)
 - Densidad de los transistores: + 35% / año
 - Tamaño del chip (dado): + 10-20% / año
 - Mejora global: 40-55% / año
- Capacidad DRAM: + 25-40% / año
(**enlenteciéndose**)
 - 8 Gb (2014), 16 Gb (2017), 32 Gb y 64 Gb (2019)

Tendencias en tecnología (II)

- Capacidad Flash: + 50-60% / año
 - 8-10X más barato/bit que DRAM
- Capacidad Discos Duros (HDs): recientemente ha bajado al 5% / año
 - No es posible mejorar la densidad, quizás pasar de 7 a 9 discos
 - 8-10X más barato/bit que Flash

La ley de Moore (1965)

- La industria podrá poner el doble de transistores en un chip cada 18-24 meses



Se trata de una ley empírica, formulada por el cofundador de Intel, Gordon E. Moore

- Esta "ley" se cumplió durante los años 1955-2005, y además también se cumplió la ley de Dennard, por lo que a medida que los transistores se hacían más pequeños, la potencia consumida por el circuito era constante

La ley de Dennard (1974)

- La ley de Dennard (*Dennard scaling*)
 - También conocida como MOSFET scaling, propuesta en un artículo de 1974 escrito por Robert H. Dennard (y otros)
 - Dice que conforme los transistores van escalando (haciéndose más pequeños), la densidad de potencia (potencia/área $\Rightarrow$ W/cm²) que consumen permanece constante
- Ejemplo de lo que esto significa:
 - Supongamos una reducción lineal en el tamaño de los transistores en un factor de $\times$
 - Eso significa una reducción en el área ocupada por cada transistor igual de $\times^2$
 - Eso implica que podré poner $\times^2$ más transistores
 - La ley de Dennard predice que la potencia disipada por cada transistor disminuye también en un factor de $\times^2$ (gracias a disminuir el voltaje y la corriente)
 - $\Rightarrow$ El chip consume lo mismo mientras que sea del mismo tamaño

Consecuencias Dennard & Moore

- ¿Qué pasa cuando el tamaño característico del transistor (*feature size*) se reduce por un factor de x ?
 - El número de transistores en el chip aumenta por x^2
 - El *clock rate* (frecuencia de funcionamiento) podía mejorar hasta $\sim x$, porque los transistores pueden cambiar de estado más rápido
 - Además, el tamaño del dado del chip también podía aumentar (no siempre lo hacía), es decir hasta $\sim x$
 - ¡El rendimiento “bruto” del chip podía aumentar hasta $\sim x^4$!
 - hasta $\sim x$ por la frecuencia
 - x^3 por tener más transistores (mayor densidad y dado) dedicados a
 - **Paralelismo**: ILP y vectorial
 - **Localidad**: cachés
 - Gracias a las mejoras en la arquitectura, la mayoría de los programas se ejecutaban más rápido isin cambiarlos!
- ⇒ Los procesadores (CPUs) doblaban su rendimiento cada 18-24 (aproximadamente)

Problemas (*Walls*)

- Al reducirse el tamaño de los transistores (**5 nm** actualmente) y mantenerse constante el tamaño del chip en torno a 2-4 cm², caben más transistores y memoria caché en un procesador
- Sin embargo, aparecen los siguientes problemas:
 - Potencia consumida
 - Bajo paralelismo a nivel de instrucciones (ILP)
 - Memoria: latencia y ancho de banda
 - Fiabilidad

Potencia y energía

- La **potencia** (eléctrica) es la proporción por unidad de tiempo con la que la energía es transferida o absorbida por un circuito eléctrico
 - Unidades: **Vatios** ($W = J / s$)

$$Potencia_{dinámica} \propto 1/2 \times Carga_capacitiva \times V^2 \times Frecuencia_cambio$$

$$Potencia_{estática} \propto 1/2 \times Corriente_{estática} \times V$$

- La **energía** es el consumo (gasto) de potencia durante un periodo de tiempo
 - Unidades: **Julios** ($J = W \times s$) ó kWh (kW gastados en 1 hora)

$$Energía_{dinámica} \propto 1/2 \times Carga_capacitiva \times V^2 \times f \times tiempo$$

Problema de la potencia

- Componentes de la potencia:

$$Potencia_{dinámica} \propto 1/2 \times Carga_capacitiva \times V^2 \times Frecuencia_cambio$$

$$Potencia_{estática} \propto 1/2 \times Corriente_{estática} \times V = 1/2 \times Corriente_{fuga} \times NTransistores \times V$$

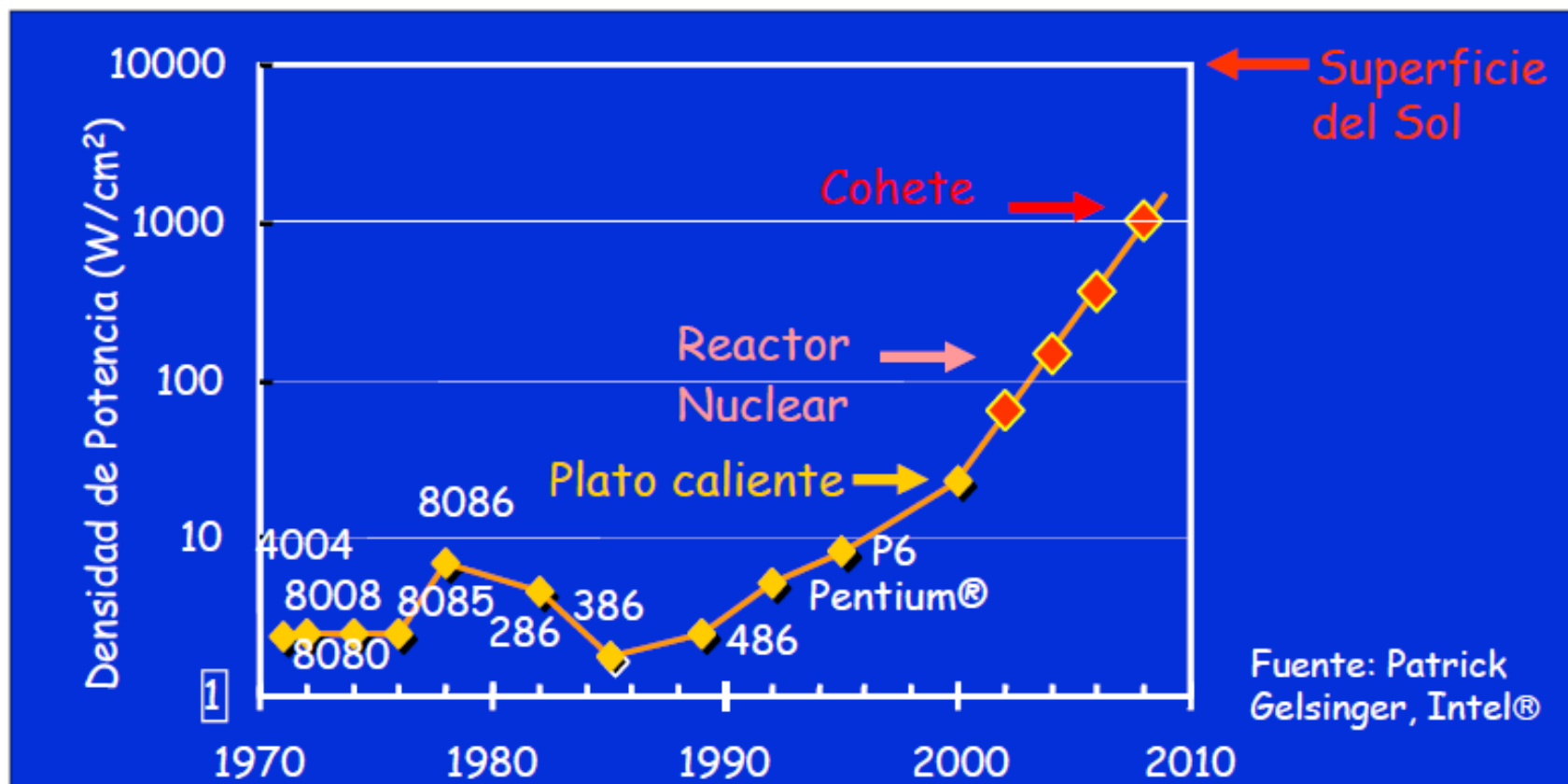
- Inicialmente, solo se tenía en cuenta la **potencia dinámica** ya que la componente **estática** era casi despreciable
- Al no poder bajar el voltaje mucho más de **$\approx 0.5\text{ V}$** la densidad de potencia empezó a aumentar (se dejó de cumplir la ley de Dennard):
 - $P_{dinámica}$ $\uparrow$ porque se sigue aumentando la frecuencia
 - $P_{estática}$ $\uparrow$ porque el número de transistores sigue creciendo
 - En la actualidad, la potencia estática puede estar en torno al 25-50% de la potencia total

Potencia y temperatura

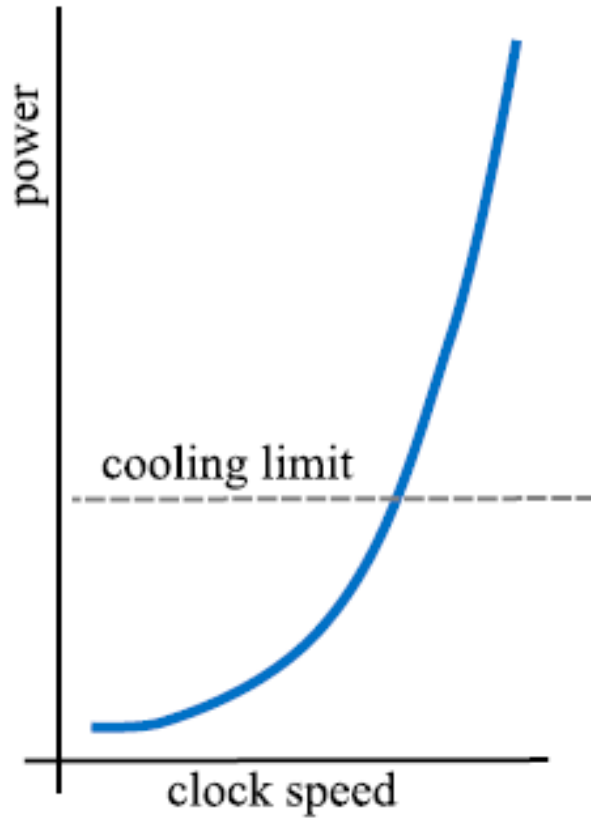
- **Potencia** y **temperatura** están directamente relacionadas
 - El procesador disipa potencia que se emite en forma de calor cada segundo. Esta potencia incrementa la temperatura del procesador
 - Hay que evitar que la temperatura se incremente demasiado disipando la energía emitida
- Potencia de diseño térmico (TDP, thermal design power)
 - Caracteriza la potencia máxima disipada por el procesador suponiendo un uso sostenido (a máximo rendimiento)
 - Utilizado como objetivo para la fuente de alimentación y el sistema de refrigeración

Problema de la densidad de potencia

- La potencia es disipada en forma de calor
- La **densidad de potencia** (W/cm^2) es proporcional a la temperatura. Determina la existencia de puntos calientes
 - Intel 80386 disipaba $\sim 2 \text{ W}$
 - Intel Core i7 a 3.3 GHz disipa $\sim 130 \text{ W}$



Problema de la temperatura



Nitrógeno
líquido para
CPU con
overclocking



Proyecto Natick
de Microsoft:
datacenter en
el océano



Las soluciones de “cooling” para altas frecuencias o son caras o no son prácticas

Cómo reducir la potencia

- Técnicas habituales para reducir la potencia disipada:
 - Reducir la frecuencia del reloj dinámicamente para limitar el consumo de energía
 - Reducir el voltaje dinámicamente cuando se reduce la frecuencia
 - Apagado selectivo de zonas (p.ej.: núcleos) del procesador
 - Estados de bajo consumo de energía en memorias, discos duros...

Eficiencia energética

- Estando relacionadas las métricas de potencia y la energía, la métrica correcta para comparar la **eficiencia energética** de diferentes procesadores es la **energía**
- En concreto, el consumo de energía está relacionado con la realización de una determinada tarea en un tiempo determinado con un procesador determinado
- **Ejemplo:** un procesador A podría tener un 20% más de potencia disipada (watts) que el procesador B, pero si A ejecuta la tarea en un 70% del tiempo de B, el consumo de energía será: **$1,2 \times 0,7 = 0,84$** lo que representa un **16%** de mejora energética para el procesador A

Rendimiento por potencia disipada

- Otra métrica para medir la eficiencia energética es el **rendimiento por vatio disipado** (*FLOPs/watt*)
- **Objetivo:** 1 ExaFlop con 20 MWatts (~ 50 GFLOPs/Watt)
 - Central nuclear: 50-1000 MWatts
 - Central hidroeléctrica: 500-20.000 MWatts
- Datos históricos:
 - UNIVAC I: 0.015 FLOPs/Watt
 - GPUs: Kepler K80: 9,8 GFLOPs/Watt
Pascal chips (P100): 17,6 GFLOPs/Watt
 - CPUs: Knights Landing: 5,8 GFLOPs/Watt
Xeon < 5 GFLOPs/Watt

https://en.wikipedia.org/wiki/Performance_per_watt

Problema: Aumentar el ILP

- $\text{Tiempo Ejec.} = \text{NI} \times \text{CPI} \times 1/\text{Freq_reloj}$
- Cómo puedo disminuir el CPI
 - Aumentar el bus de datos
 - Segmentación (pero a partir de 10 etapas aparecen muchos conflictos)
 - Replicar cauce: ejecución superescalar
 - Ejecución fuera de orden (OoO) y especulativa (predicción de saltos)
 - Hay que ser muy eficiente para resolver los frecuentes riesgos de datos y de control
 - Mucho hardware → consume mucha energía

Problema: Memoria

- La mejora de las CPUs es más rápida que la mejora en el tiempo de acceso a memoria RAM: cada vez las instrucciones pierden más ciclos esperando a memoria (**latencia**)
- La ejecución fuera de orden (OoO) solo puede mitigar parcialmente este problema
- La mejora en el ancho de banda de memoria (número de canales entre la CPU y la memoria) tampoco puede solucionar este problema

Problema: Fiabilidad

- **Tolerancia a fallos.** Los componentes son muy pequeños y cada vez son más sensibles a partículas cargadas del ambiente, por lo que pueden fallar (cambiar 0 por 1 y viceversa)
- Con la miniaturización: cada vez es más complicado que no falle una parte del circuito
- En la actualidad, es común tener memorias y caches con soluciones ECC (*Error Correction Code*)
- A su vez, cada vez es más complicado comprobar un nuevo diseño para la (micro)arquitectura de un procesador

Problema: Fiabilidad

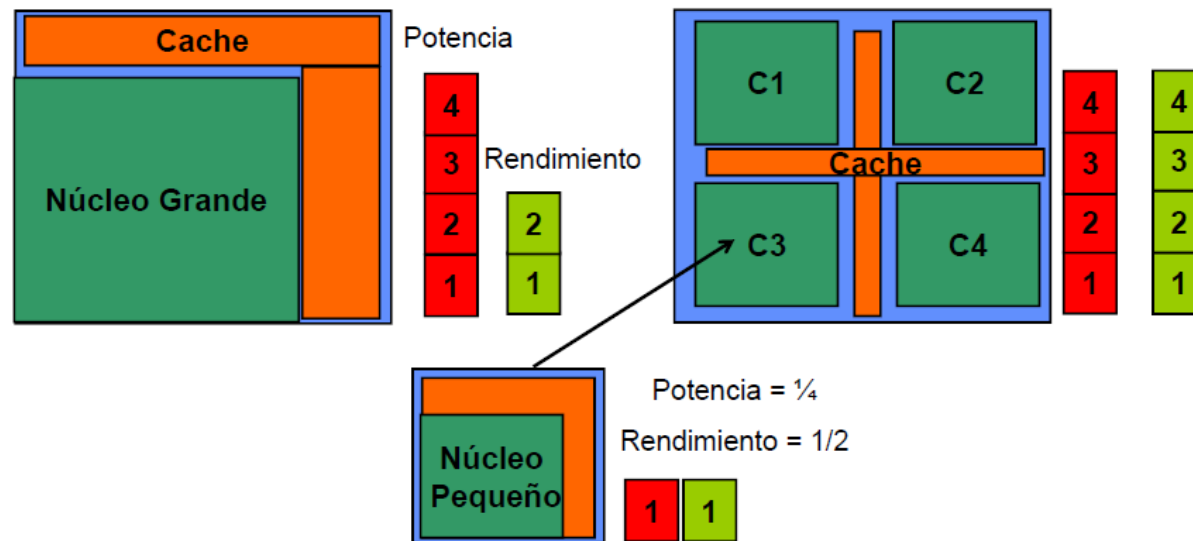
- Métricas
 - Tiempo medio hasta un fallo (MTTF)
 - Tiempo medio para reparaciones (MTTR)
 - Tiempo medio entre fallos (MTBF) = $MTTF + MTTR$
 - Disponibilidad = $MTTF / MTBF$

Otros problemas del diseño monolítico

- Adicionalmente, los procesadores presentan otros problemas importantes:
 - **Clock skew (sesgo de reloj)**: Un procesador es un circuito que funciona síncronamente. Los transistores reciben y alteran la señal de reloj por lo que, al haber muchos transistores, puede haber modificaciones perceptibles en la señal de reloj
 - **Problemas eléctricos**: impedancia y capacitancia
- Esto da lugar al fenómeno conocido como ***Dark silikon***: parte del procesador que no podemos utilizar por dichos problemas

Solución: CMPs (*Chip Multiprocessors*)

- **Potencia:** Con núcleos más pequeños se puede reducir la frecuencia y voltaje para conseguir un mismo rendimiento → Varios núcleos pueden alcanzar más rendimiento/watt
- **ILP:** El CMP tiene más transistores *útiles* que el diseño de un único procesador lo que permite obtener un mejor rendimiento
- **Memoria:** Se usan nodos NUMA, con localidad y menor latencia. Además, cada núcleo usa su caché privada (L1 y L2, habitualmente)
- **Fiabilidad:** También mejoran la fiabilidad del chip por tener redundancia entre los núcleos, y usar un mismo diseño para los diferentes núcleos



Solución: CMPs (*Chip Multiprocessors*)

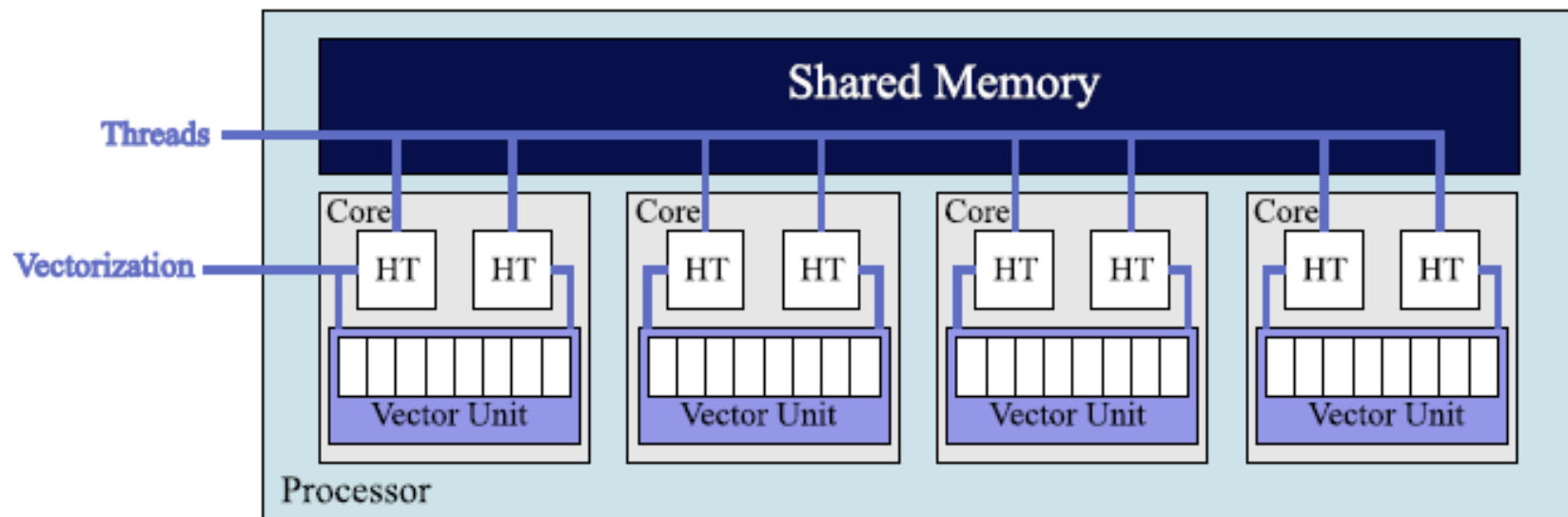
- **Ventajas:**

- En los CMP cada núcleo cuenta con su propio reloj, **desaparece** el sesgo de reloj, y se **mitigan** los problemas eléctricos
- Es más fácil verificar y validar el funcionamiento de varios chips simples que el de uno muy complejo (*time-to-market*)
- **Redundancia y tolerancia a fallos.** Al tener los CMPs varios núcleos, podemos dedicar uno o varios de ellos a comprobar lo que hacen los otros (redundancia)

- Por tanto, se consigue un mayor rendimiento con menos consumo energético y teniendo que disipar menos calor

CMPs: explotando el paralelismo

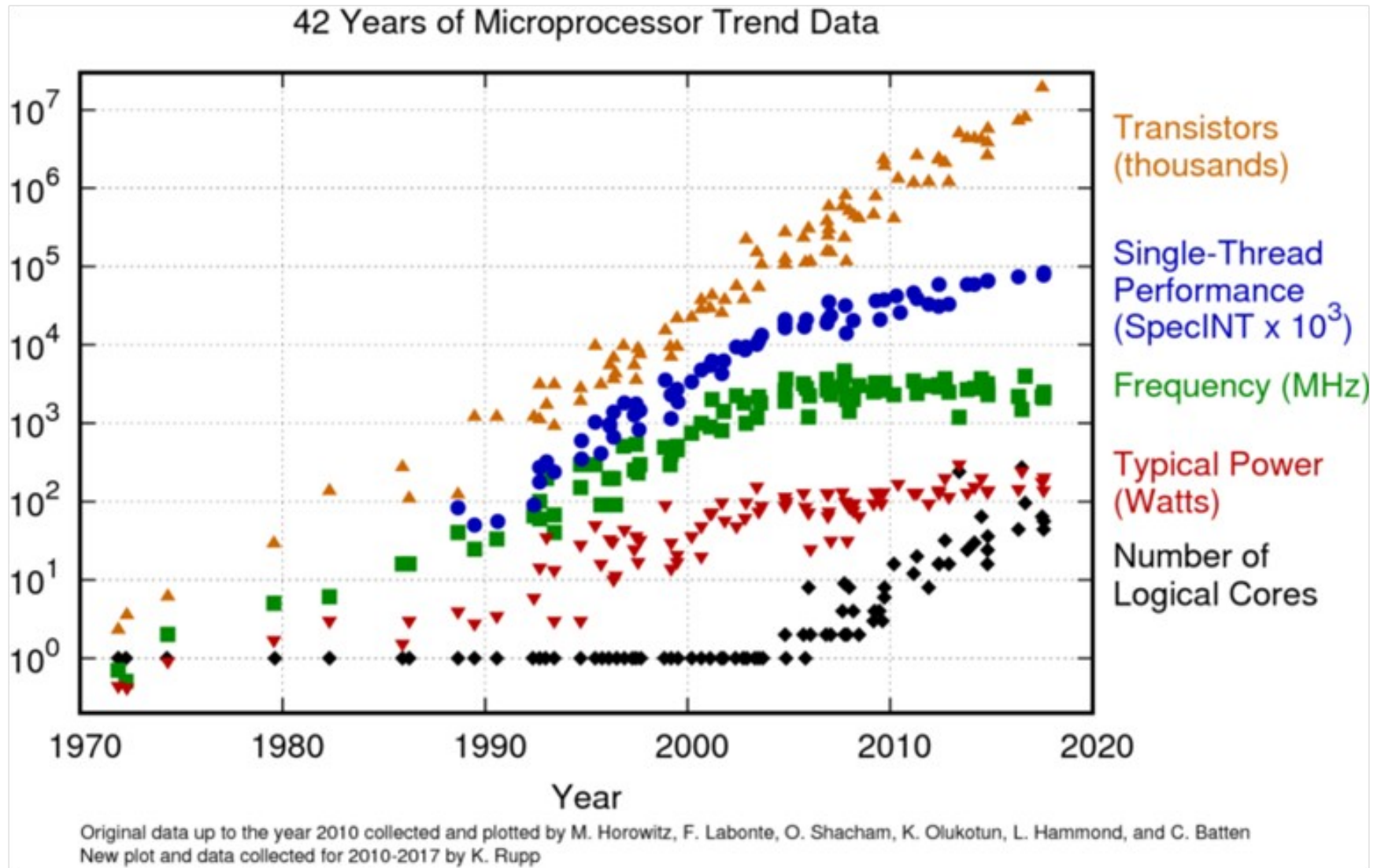
- A varios niveles:
 - ILP, especulación y réplica de cauces
 - Vectores: Una instrucción sobre múltiples datos
 - Núcleos / hilos: Múltiples instrucciones sobre múltiples datos



La ley de Moore reinterpretada

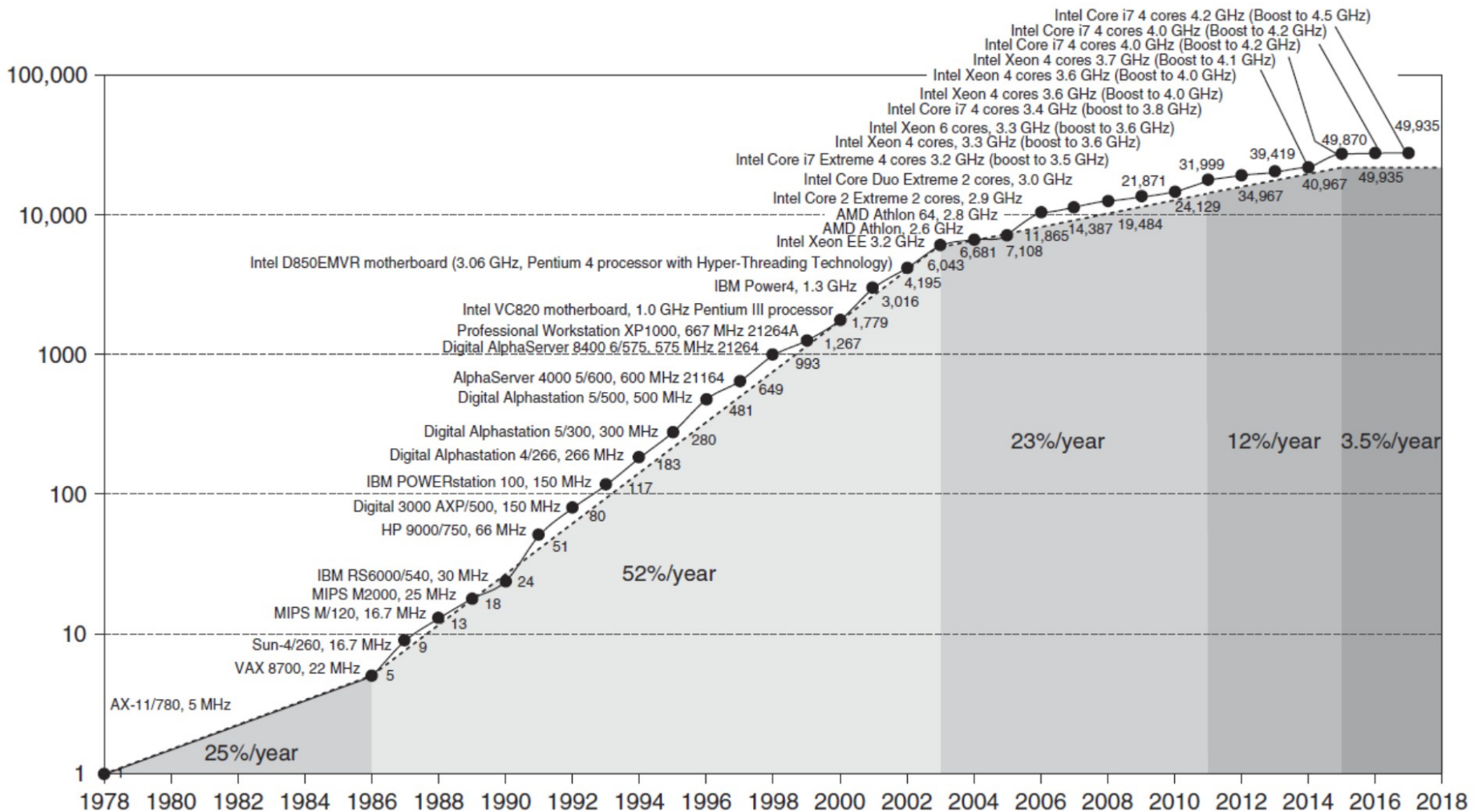
- **El número de núcleos por chip se puede doblar cada 2 años**
- La frecuencia de reloj ya no se va a incrementar **(posiblemente decrementar)**
- Es necesario manejar el paralelismo:
 - dentro del núcleo (*intra-core*): ILP y **unidades vectoriales**
 - entre los diversos núcleos (*inter-core*): **Múltiples núcleos** en el procesador
- **Necesitamos manejar procesadores con miles y millones de hilos de forma concurrente**

40 años de microprocesadores



Fuente: karlrupp.net

Rendimiento de un único procesador



Copyright © 2019, Elsevier Inc. All rights reserved.

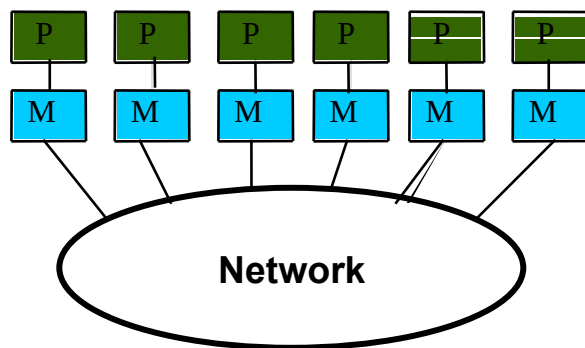
Agenda

- 1.1 Aspectos tecnológicos: Leyes de Moore y de Dennard
- 1.2 Clasificación de Flynn. Principales figuras de mérito
- 1.3 Aspectos paralelos: Leyes de Amdahl y Gustafson
- 1.4 De sistemas empujados a supercomputadores

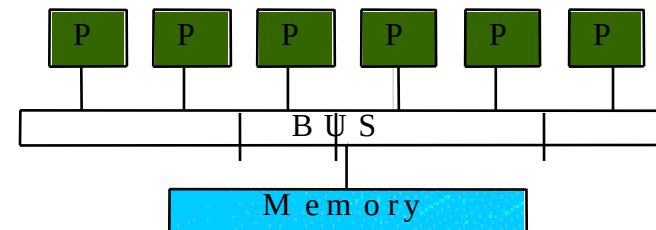
Clasificación de arquitecturas

- Taxonomía de Michael J. Flynn (1972):
 - **SISD**, *Single Instruction Single Data*
 - Sistemas monoprocesador basados en microprocesadores *monocore* como PCs antiguos (no CMPs)
 - **SIMD**, *Single Instruction Multiple Data*
 - Procesadores vectoriales, GPUs
 - **MISD**, *Multiple Instruction Single Data*
 - Ninguna máquina comercial existente
 - **MIMD**, *Multiple Instruction Multiple Data*
 - Arquitectura adoptada por los computadores de hoy en día
 - Procesadores multinúcleo, clusters, servidores, supercomputadores...

MIMD: Memoria compartida o distribuida



Memoria distribuida – cada procesador tiene su propia memoria local. Para comunicar datos, se necesita usar explícitamente instrucciones de paso de mensajes (ejemplo: clúster)



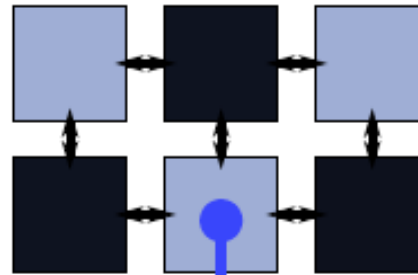
Memoria compartida: Un único espacio de direcciones al que todos los procesadores tienen acceso. La comunicación de datos se realiza a través de operaciones típicas de lectura/escritura de datos. (ejemplo: arquitectura multinúcleo)

Para el acceso a memoria:

- Bus, Crossbar
- Red directa (ej, malla)

Niveles de paralelismo explícito

CLUSTER COMPUTING
in distributed memory



```
MPI_Sendrecv(data, k,
MPI_DOUBLE, data2,
... );
```

MULTITHREADING
in shared memory



```
#pragma omp parallel for
for (j = 0; j < m; j++)
  ComputeSubset(j);
```

VECTORIZATION
of floating-point math



```
#pragma omp simd
for (i = 0; i < n; i++)
  A[i] += B[i];
```

Paralelismo

- Clases de paralelismo en aplicaciones:
 - Paralelismo de datos (DLP)
 - Paralelismo de tareas (TLP)
- Clases de paralelismo en la arquitectura:
 - Paralelismo de instrucciones (ILP, implícito o explícito)
 - Arquitecturas vectoriales/Unidades de Procesamiento Gráficos (GPUs)
 - Paralelismo de hilos (CMPs, varios hilos por núcleo, varios chips...)

Principales figuras de mérito (I)

- Métricas típicas de rendimiento:
 - Tiempo de ejecución
 - Productividad (*throughput*)
- Tiempo de ejecución
 - Tiempo del reloj de pared: incluye todos los *overheads* del sistema
 - Tiempo de CPU: solo tiempo de cálculo
- Aceleración de X con respecto a Y
 - $\text{Tiempo_ejecución_Y} / \text{Tiempo_ejecución_X}$
- Benchmarks
 - Kernels (por ejemplo, multiplicación de matrices)
 - Programas de juguete (por ejemplo, sorting)
 - Benchmarks sintéticos (por ejemplo, Dhrystone)
 - Benchmarks suites (por ejemplo, SPEC06fp, TPC-C)

Principales figuras de mérito (I-bis)

- La ecuación del rendimiento del procesador

$$CPU\ time = CPU\ clock\ cycles\ for\ a\ program \times Clock\ cycle\ time$$

$$CPU\ time = \frac{CPU\ clock\ cycles\ for\ a\ program}{Clock\ rate\ time}$$

$$CPI = \frac{CPU\ clock\ cycles\ for\ a\ program}{Instruction\ count}$$

$$CPU\ time = Instruction\ count \times Cycles\ per\ instruction \times Clock\ cycle\ time$$

$$CPU\ time = \frac{Instructions}{Program} \times \frac{Clock\ cycles}{Instruction} \times \frac{Seconds}{Clock\ cycles} = \frac{Seconds}{Program}$$

Principales figuras de mérito (I-bis)

- Instrucciones diferentes tienen diferentes CPIs

$$CPU \text{ clock cycles} = \sum_{i=1}^n IC_i \times CPI_i$$

$$CPU \text{ time} = \left(\sum_{i=1}^n IC_i \times CPI_i \right) \times \text{Clock cycle time}$$

Principales figuras de mérito (II)

Ancho de banda (a veces productividad)

- Trabajo total realizado en un tiempo determinado
- 32.000-40.000X de mejora para procesadores
- 300-1.200X de mejora para la memoria y HDs

- Latencia o tiempo de respuesta

- Tiempo entre el comienzo y el fin de un evento
- 50-90X de mejora para procesadores
- 6-8X de mejora para la memoria y Hds

- TDP (Thermal Design Power)

- Potencia máxima que puede llegar a disipar funcionando correctamente

Principales figuras de mérito (III)

- **Rendimiento pico (Peak Performance o PP)** en operaciones en coma flotante. Se suele medir en GFLOPS/TFLOPS (Giga/Tera Floating point Operations Per Second), que son Miles/Millones de Millones de instrucciones en coma flotante ejecutadas por segundo. Se calcula como:

$$\text{PeakPerf} = \text{Freq máxima} * \text{Nº núcleos físicos} * \text{OPS/Núcleo/Ciclo}$$

- Frecuencia max de Reloj: La velocidad de reloj máxima que puede alcanzar el dispositivo, que depende de las características de diseño del hardware, como las puertas/reloj del transistor, el consumo de energía y la tecnología del proceso.
- Número de Núcleos: Para las CPU, este es el número de núcleos de procesamiento, y para las GPU, el número de Unidades de Cómputo (también conocidas como *Streaming Multiprocessors*).
- OPS/Núcleo/Ciclo: El número de operaciones de punto flotante por núcleo que se pueden procesar en un solo ciclo. Varía según según el tipo de datos (FP64, FP32 o F16). Para los procesadores con una instrucción de multiplicación-acumulación fusionada (FMA), tanto la multiplicación como la acumulación cuentan como una "op", por lo que la FMA realiza 2 ops.

Principales figuras de mérito (IV)

- **Rendimiento pico alcanzable (Achievable Peak Performance)** es el "Máximo PP Alcanzable" (MAF), es decir, son los FLOP máximos alcanzables en situaciones de carga de trabajo realistas elegidas para maximizar la eficiencia arquitectónica.
- El principal problema es la frecuencia de reloj, ya que el chip consume una cantidad significativa de energía y hace que la frecuencia de reloj "real" sea inferior a la frecuencia máxima. Los procesadores modernos cuentan con sofisticados algoritmos de gestión de energía que detectan el consumo del dispositivo, elevando la frecuencia de reloj a la frecuencia máxima cuando es posible y reduciéndola en las fases de computación densa. Estas transiciones son gestionadas automáticamente por el hardware y pueden ocurrir con frecuencia (es decir, en microsegundos).
- Por ello, para saber lo "bueno" que es el rendimiento de un algoritmo lo habitual es compararlo al *Achievable Peak Performance* (APP)

Principales figuras de mérito (IV)

- Tamaños comunes suelen ser millones, billones, trillones (ingleses)

Mega	Mflop/s = 10^6 flop/sec	Mibyte (MiB) = $2^{20} = 1.048.576 \sim 10^6$ bytes (MB)
Giga	Gflop/s = 10^9 flop/sec	Gibyte (GiB) = $2^{30} \sim 10^9$ bytes (GB)
Tera	Tflop/s = 10^{12} flop/sec	Tibyte = $2^{40} \sim 10^{12}$ bytes
Peta	Pflop/s = 10^{15} flop/sec	Pibyte = $2^{50} \sim 10^{15}$ bytes
Exa	Eflop/s = 10^{18} flop/sec	Eibyte = $2^{60} \sim 10^{18}$ bytes
Zetta	Zflop/s = 10^{21} flop/sec	Zibyte = $2^{70} \sim 10^{21}$ bytes
Yotta	Yflop/s = 10^{24} flop/sec	Yibyte = $2^{80} \sim 10^{24}$ bytes

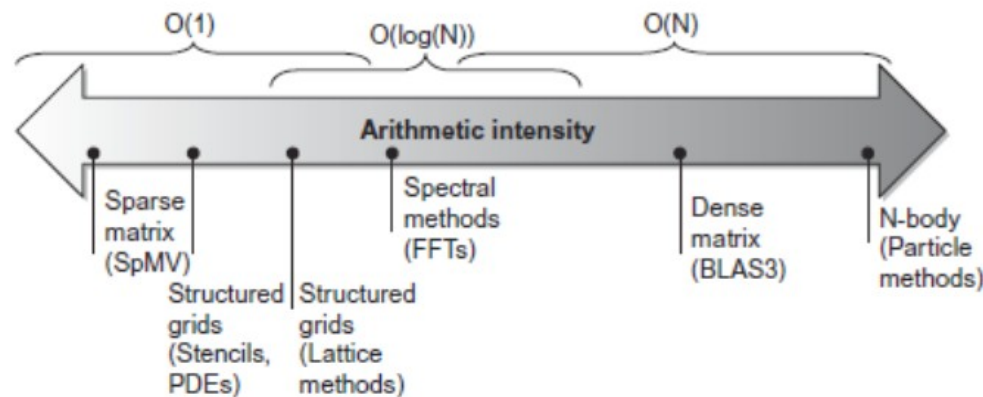
- Como referencia, el ordenador más rápido del mundo es el Frontier, en el Oak Ridge National Laboratory (ORNL) en USA, y es el primer ordenador en romper el techo de Exaflops, con un rendimiento de 1,206 Exaflop/s. Tiene 8.699.904 núcleos en total, un consumo energético de 22,78 MW y tiene una Gigabit Ethernet para la transferencia de datos.
- Lista actualizada en www.top500.org

Principales figuras de mérito (V)

- Otros aspectos relevantes también son:
 - Frecuencia del reloj
 - Número de transistores
 - Tamaño de la Caché (y niveles)
 - Cantidad de memoria direccionable
 - Ancho de banda del bus a memoria
 - Número de núcleos/hilos por chip
 - Tamaño de la litografía
 - Tamaño de los operandos

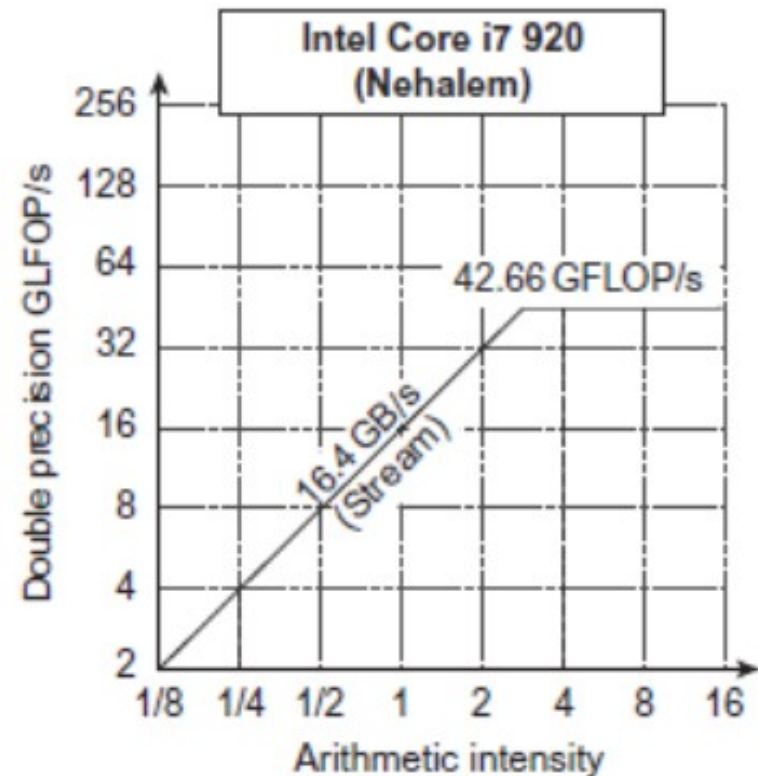
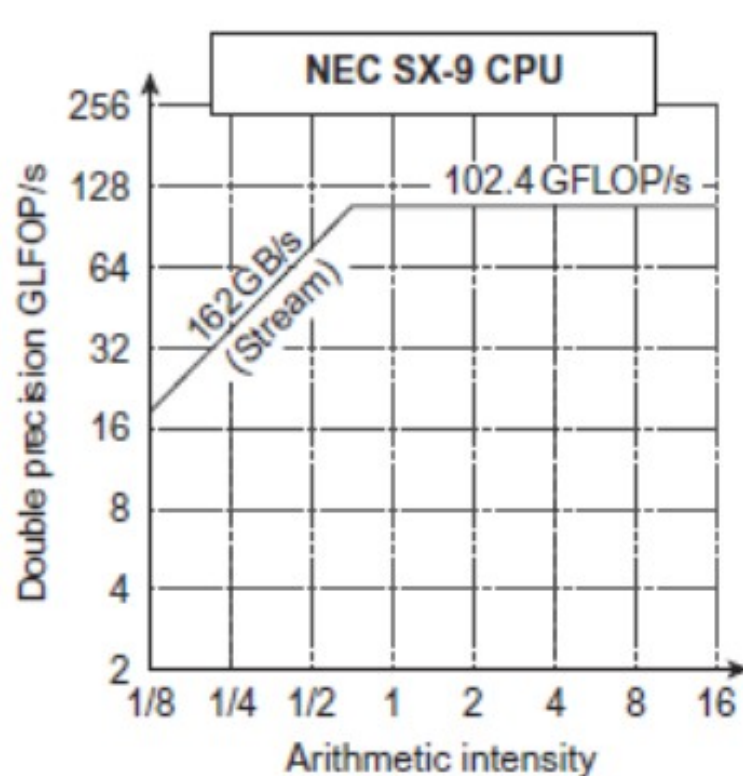
Modelo de rendimiento *Roofline* (I)

- Idea básica:
 - Poner el rendimiento pico en coma flotante como función de la intensidad aritmética
 - Ofrecer en un único valor el rendimiento de punto flotante y el rendimiento de memoria para una máquina de destino
- Intensidad aritmética
 - Operaciones en coma flotante por byte (o operando) transferido de memoria (leído o escrito)
 - Es una propiedad (de la implementación) del algoritmo



Modelo de rendimiento *Roofline* (II)

- GFLOPs/sec (alcanzables) = Mínimo (Peak Memory BW $\times$ Arithmetic Intensity, Peak Floating Point Perf.)



Agenda

- 1.1 Aspectos tecnológicos: Leyes de Moore y de Dennard
- 1.2 Clasificación de Flynn. Principales figuras de mérito
- 1.3 Aspectos paralelos: Leyes de Amdahl y Gustafson
- 1.4 De sistemas empujados a supercomputadores

Paralelismo: Speedup

- Ganancia en prestaciones (speedup):

- Indica la medida en la que se incrementan las prestaciones al ejecutar una aplicación en paralelo en un sistema con varios procesadores frente a su ejecución uniprocador
- Permite evaluar un computador paralelo o la implementación de la aplicación paralela

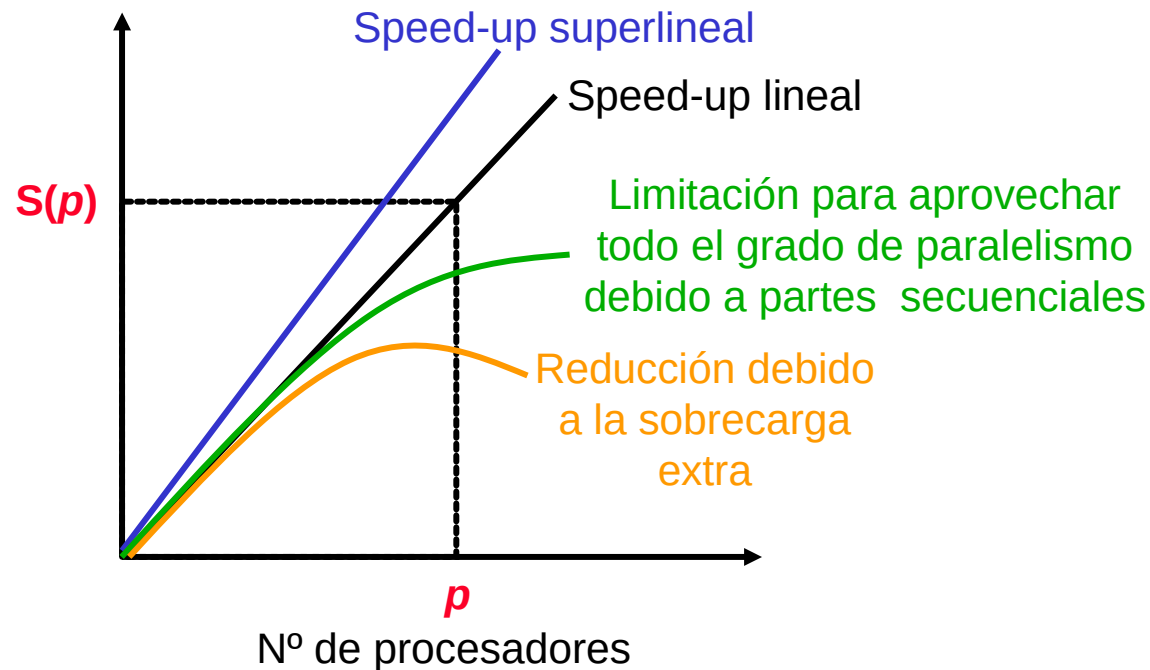
$$S(p) = \frac{\text{Performance}(p)}{\text{Performance}(1)} = \frac{T_o}{T_p(p)}$$

- $S(p)$: Aceleración o *speed-up* con p procesadores
- $\text{Performance}(p)$: velocidad con p procesadores (en FLOPS, MIPS...)
- T_o : tiempo de ejecución del programa secuencial (la mejor versión secuencial)
- $T_p(p)$: tiempo de ejecución del programa con p procesadores, puede verse como la suma de dos componentes:
 - » $T_c(p)$: tiempo de cómputo en paralelo de las tareas de la aplicación
 - » $T_o(p)$: tiempo de penalización o sobrecarga debido a comunicación/sincronización entre procesos, creación/terminación de los procesos, y operaciones añadidas en la versión paralela no presentes en la secuencial

Paralelismo: Speedup

- Escalabilidad:

- Evaluada a partir de la representación de la ganancia (speedup) en función del número de procesadores.



Paralelismo: Límite teórico superior

- Todos los programas paralelos tienen:
 - Secciones paralelas
 - Secciones secuenciales
- Las secciones secuenciales son debidas:
 - Varios procesadores realizan el mismo trabajo (duplicación)
 - Sólo un procesador está haciendo algo útil, mientras que el resto espera a que éste acabe sin hacer nada útil
- Las secciones secuenciales limitan la efectividad del rendimiento obtenido por el paralelismo
 - La aceleración obtenida está en función de la cantidad de código secuencial que tiene una aplicación
 - “Sólo” hay aceleración perfecta si no hay parte secuencial
- La **ley de Amdahl** formaliza esta idea

Ley de Amdahl (I)

- La ley de Amdahl permite calcular el speedup conseguido gracias a una mejora que afecta a una parte de un proceso
 - Aplicándola a la paralelización, nos da una cota superior de la mejora que podemos obtener usando un cierto número de procesadores **manteniendo el trabajo total constante** (escalado **fuerte**)

T_o : Tiempo original

T_p : Tiempo mejorado

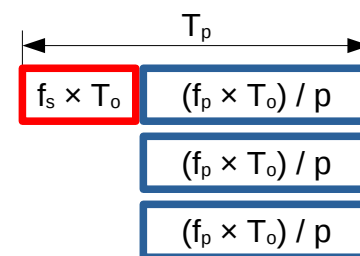
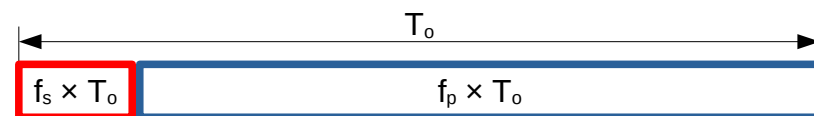
f_p : Fracción mejorable (paralelizable) $= 1 - f_s$

f_s : Fracción no mejorable (secuencial) $= 1 - f_p$

p : Factor de mejora (número de procesadores)

$$T_p = f_s \times T_o + \frac{f_p \times T_o}{p}$$

$$S_{strong} = \frac{T_o}{T_p} = \frac{T_o}{(1 - f_p) \times T_o + \frac{f_p \times T_o}{p}} = \frac{1}{1 - f_p + \frac{f_p}{p}}$$



Ley de Amdahl (II)

- Paralelización ideal (todo el proceso es paralelizable):

$$T_p = f_s \times T_o + \frac{f_p \times T_o}{p} = \frac{T_o}{p}$$

$$\begin{matrix} f_p = 1 - f_s = 1 \\ f_s = 1 - f_p = 0 \end{matrix} \Rightarrow S_{strong} = \frac{1}{1 - f_p + \frac{f_p}{p}} = \frac{1}{\frac{1}{p}} = p$$

- Consecuencia de la ley de Amdahl: incluso utilizando un número infinito de procesadores, el speedup máximo que se puede obtener está limitado por la parte secuencial (escalado fuerte)

$$S_{strong} = \frac{1}{1 - f_p + \frac{f_p}{p}} \Rightarrow \lim_{p \rightarrow \infty} S_{strong} = \frac{1}{1 - f_p} = \frac{1}{f_s}$$

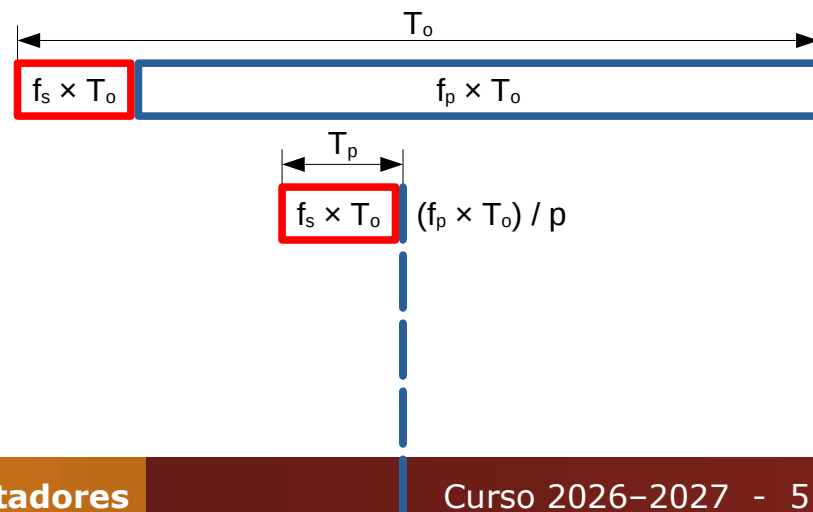
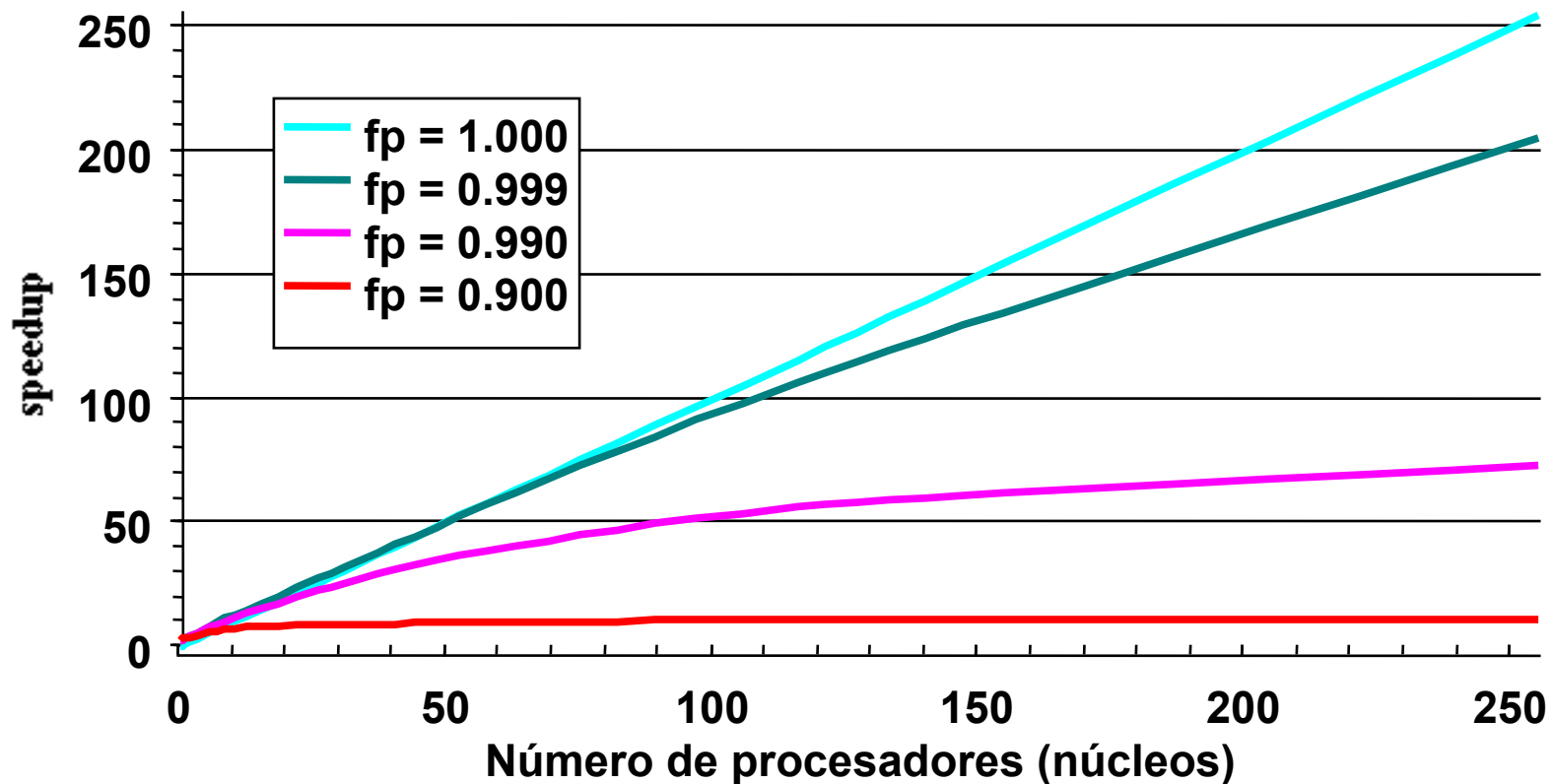


Ilustración de la Ley de Amdahl

- Una pequeña fracción de código secuencial limita enormemente el rendimiento paralelo alcanzable



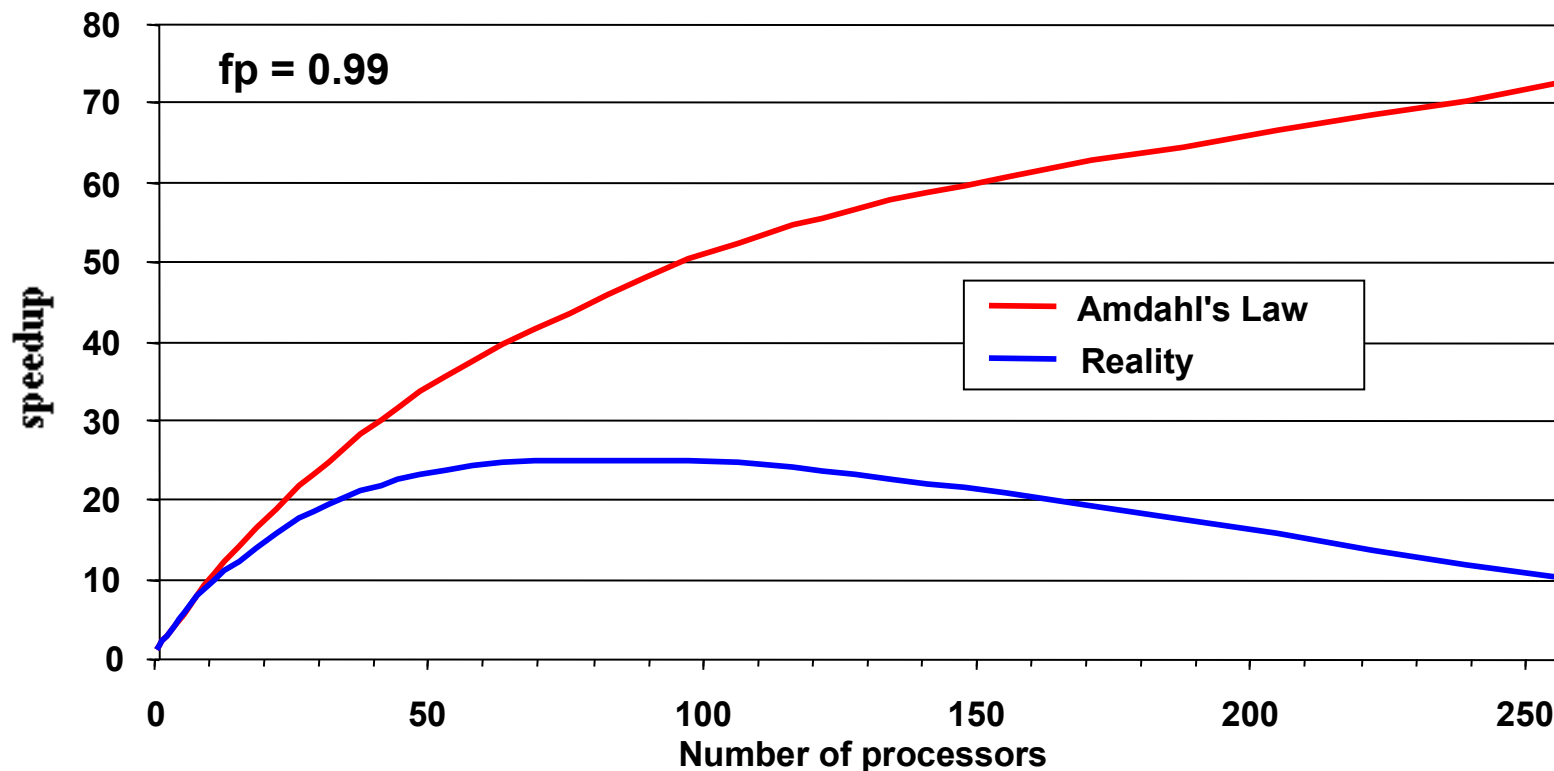
Sobrecoste del Paralelismo

- Aunque tengamos un problema bastante paralelo, hay que tener cuidado con el sobrecoste de:
 - Comenzar un hilo o proceso
 - Comunicar los datos compartidos
 - Sincronizar los datos
 - Computación extra (redundante) que hay que hacer
- Cada problema de los anteriores puede estar en el rango de milisegundos (=millones de flops)

La Ley de Amdahl vs la realidad

- La ley de Amdahl da una cota superior. Pero debido a las comunicaciones, el *speedup* real puede ser mucho menor.
- El tiempo añadido por esto (T_c) suele crecer con el número de hilos (p).

$$T_p = f_s \times T_o + \frac{f_p \times T_o}{p} + T_c$$



Escalado débil (ley de Gustafson-Barsis)

- Los resultados que hemos visto al aplicar la ley de Amdahl nos indican que, **manteniendo constante la cantidad total de trabajo** (escalado fuerte), es muy difícil obtener buenos speedups para un número elevado de procesadores
 - Con solo un 5% no paralelizable, el speedup máximo es de 20 aunque utilicemos cientos de miles de hilos
 - De esta forma, los beneficios de la computación paralela serían muy limitados y, por ejemplo, no valdría la pena diseñar procesadores con muchos cores
- Afortunadamente, en muchos problemas es razonable suponer que queremos **aumentar la cantidad de trabajo a la vez que aumentamos el número de procesadores**
 - Ejemplos: procesar imágenes con mayor resolución, usar redes neuronales más complejas, aumentar el número de ATMs que puede gestionar un banco...
 - El escalado débil permite que podamos abordar problemas mas grandes sin aumentar el tiempo de proceso

Ley de Gustafson (I)

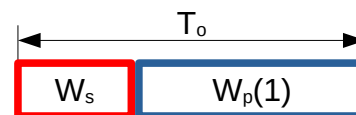
- La ley de Gustafson permite calcular el speedup conseguido al paralelizar cuando **la cantidad de trabajo paralelizable aumenta proporcionalmente con el número de procesadores**.

$T_o = T_s(1) = T_p(1)$: Tiempo original

$W_s = T_o \times f_s$: Trabajo no paralelizable (constante)

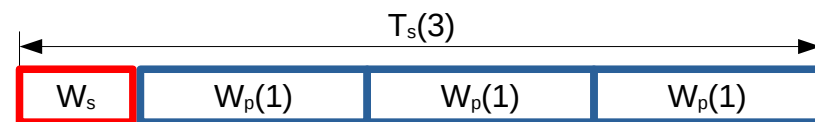
$W_p(1) = T_o \times f_p$: Trabajo paralelizable original

$W_p(p) = W_p(1) \times p$: Trabajo paralelizable (crece)



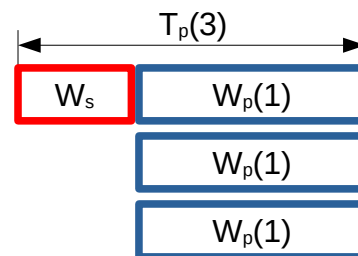
Tiempo secuencial para p veces más trabajo:

$$T_s(p) = W_s + W_p(p) = W_s + W_p(1) \times p$$



Tiempo paralelo para p veces más trabajo usando p hilos:

$$T_p(p) = W_s + W_p(p) = W_s + \frac{W_p(1) \times p}{p} = W_s + W_p(1)$$



$$\begin{aligned} S_{weak} &= \frac{T_s(p)}{T_p(p)} = \frac{W_s + W_p(1) \times p}{W_s + W_p(1)} = \frac{(1-f_p) \times T_o + f_p \times T_o \times p}{(1-f_p) \times T_o + f_p \times T_o} = \frac{1-f_p + f_p \times p}{1-f_p + f_p} = 1 - f_p + f_p \times p \\ &= 1 - (1-f_s) + (1-f_s) \times p = f_s + (1-f_s) \times p = f_s + p - f_s \times p = p - f_s \times (p-1) \end{aligned}$$

Ley de Gustafson (II)

$$S_{weak} = 1 - f_p + f_p \times p \Rightarrow \lim_{p \rightarrow \infty} S_{weak} = \infty$$

- Consecuencia de la ley de Gustafson: utilizando un número infinito de procesadores, el speedup máximo que se puede obtener es ilimitado, si estamos dispuestos a aumentar el tamaño del problema proporcionalmente
 - ⇒ Podemos incrementar el tamaño del problema tanto como queramos, si disponemos de suficientes procesadores.
 - Más resolución de imagen, redes neuronales más complejas...
- Cambio de dirección de los objetivos de investigación hacia tratar de resolver problemas cada vez de mayor tamaño en el mismo intervalo de tiempo

Agenda

- 1.1 Aspectos tecnológicos: Leyes de Moore y de Dennard
- 1.2 Clasificación de Flynn. Principales figuras de mérito
- 1.3 Aspectos paralelos: Leyes de Amdahl y Gustafson
- 1.4 De sistemas empuotrados a supercomputadores

Principios de Computación Paralela

- Encontrar problemas con bastante paralelismo (Ley de Amdahl) o problemas cuyo tamaño aumente (Ley de Gustafson)
- Granularidad – cómo de grande debe ser cada tarea paralela
- Localidad – mover los datos cuesta más que procesarlos
- Carga balanceada – el hilo más lento limita el rendimiento
- Coordinación y sincronización – compartir datos de forma segura
- Modelado del rendimiento / depuración / ajuste

➡ Todas estas cosas hacen la programación paralela incluso más complicada que la secuencial

Clases de Computadores

- Internet of Things (IoT) / Computadores empotrados
 - Énfasis: coste económico
- Personal Mobile Device (PMD)
 - Ej.: *smartphones*, tablets, ...
 - Énfasis: eficiencia energética y tiempo real
- Desktop Computing
 - Énfasis: *price-performance* (coste/rendimiento)
- Servidores
 - Énfasis: disponibilidad, escalabilidad, productividad
- Clusters / Warehouse Scale Computers
 - Usados como "Software as a Service (SaaS)"
 - Énfasis: disponibilidad, coste/rendimiento
- Supercomputadores
 - Énfasis: velocidad en coma flotante (PFLOPs), consumo de energía, red de interconexión

Plataformas (Procesadores) Intel

General-Purpose Processors

Intel® Xeon®
Intel® Core™
Intel® Atom™, ...



Specialized Processors

Intel® Xeon Phi™
processors
and coprocessors



Computing Accelerators

Intel® VCA (x86)
Intel® Nervana™ Platform
Intel® DLIA™ (FPGAs)

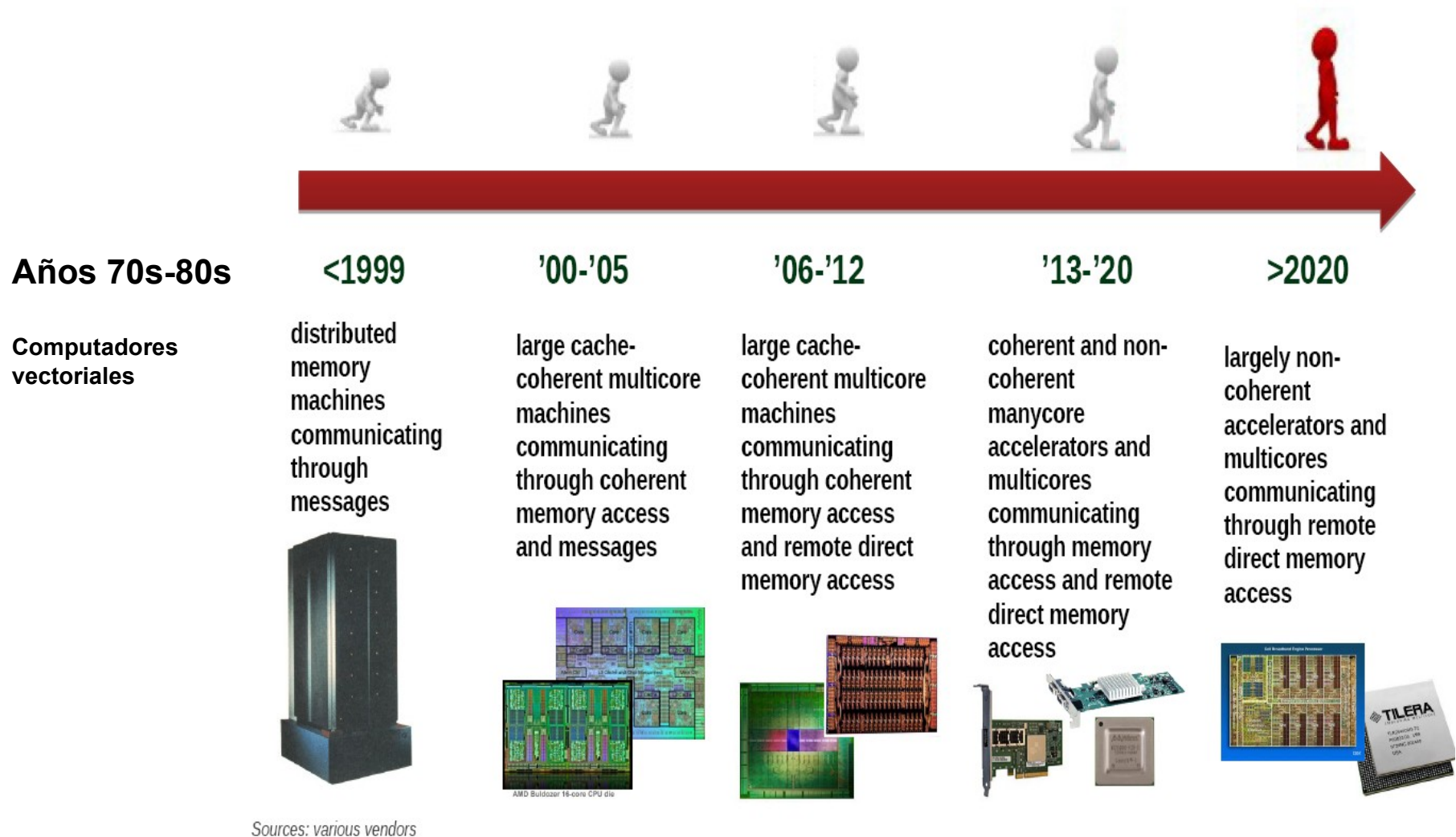


Network Interconnects

Intel® Omni-Path™
Architecture



Desarrollo de HPC (*High Performance Computing*)



Supercomputadores Vectoriales



[©Cray Research, 1976]

- Término acuñado en el Cray-1, 1976
- Unidad escalar
 - Arquitectura Carga/almacenamiento
- Extensión Vectorial
 - Registros Vectoriales
 - Instrucciones Vectoriales
- Implementación
 - Control cableado
 - UFs segmentadas (mucho)
 - Sistema de memoria multi-banco
 - Sin cache de datos
 - Sin memoria virtual

Supercomputadores

- Definición de un supercomputador:
 - La máquina más rápida del mundo en una tarea determinada
 - Cualquier máquina que cueste más de \$30M
 - Cualquier máquina diseñada por Seymour Cray
- El CDC6600 (Cray, 1964) es considerado como el primer supercomputador

MEMORANDUM

August 28, 1963

Memorandum To: Messrs. A. L. Williams
T. V. Learson
H. W. Miller, Jr.
E. R. Piore
O. M. Scott
M. B. Smith
A. K. Watson

Last week CDC had a press conference during which they officially announced their 6600 system. I understand that in the laboratory developing this system there are only 34 people, "including the janitor." Of these, 14 are engineers and 4 are programmers, and only one person has a Ph. D., a relatively junior programmer. To the outsider, the laboratory appeared to be cost conscious, hard working and highly motivated.

Contrasting this modest effort with our own vast development activities, I fail to understand why we have lost our industry leadership position by letting someone else offer the world's most powerful computer. At Jenny Lake, I think top priority should be given to a discussion as to what we are doing wrong and how we should go about changing it immediately.

TJW, Jr:jmc

T. J. Watson, Jr.

cc: Mr. W. B. McWhirter

[© IBM]

Top 500 (www.top500.org)



Rendimiento: 122 PFlop/s
Nº núcleos: 2,3 M
Energía consumida: 8,8 MW



**Summit: DOE/SC/Oak Ridge National Laboratory
No. 1 in June 2018**

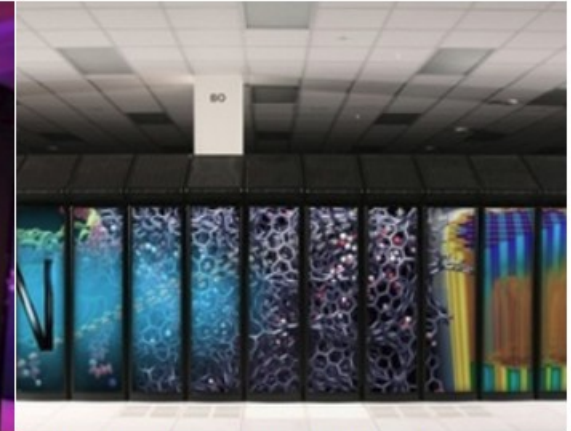
Top-500 (www.top500.org)



Sunway TaihuLight: National Supercomputing Center in Wuxi
No.1 from Jun 2016 until Jun 2017



Tianhe-2 (MilkyWay-2) : National University of Defense Technology
No.1 from Jun 2013 until Nov 2015



Titan: Oak Ridge National Laboratory
No.1 in Nov 2012



Sequoia: Lawrence Livermore National Laboratory
No.1 in Jun 2012



K Computer: RIKEN Advanced Institute for Computational Science
No.1 from Jun 2011 until Nov 2011



Tianhe-1A: National Supercomputing Center in Tianjin
No.1 in Nov 2010

Algunos problemas especialmente desafiantes

- Ciencia
 - Modelado climático global
 - Biología: genómica; plegamiento de proteínas; diseño de fármacos
 - Modelado Astrofísico
 - Química Computacional
 - Ciencias de los Materiales y Nanociencias
- Ingeniería
 - Diseño de semiconductores
 - Terremotos y modelado estructural
 - Dinámica de fluidos (diseño de aviones, barcos, etc)
 - Combustión (diseño de motores)
 - Simulación de accidentes de coches
- Negocios
 - Modelado financiero y económico
 - Procesamiento de transacciones, servicios web y motores de búsqueda
- Defensa
 - Armas nucleares: prueba por simulaciones
 - Criptografía

Lecturas adicionales de interés

The 50 Year History of the Microprocessor as Five Technology Eras. John L. Hennessy. IEEE Micro, November/December 2021.

The First Trillion-Transistor Chip: A New Design Space. Hsien-Hsin Sean Lee, Computer Architecture Today, ACM SIGARCH, Aug 29, 2019.

A New Golden Age for Computer Architecture. John L. Hennessy and David A. Patterson, Communications of the ACM, February 2019, vol. 62, no. 2.

Créditos

- Intel Xeon Phi Processor High Performance Programming, Knights Landing Edition by James Reinders, Jim Jeffers and Avinash Sodani, copyright 2016, published by Morgan Kaufmann, ISBN 978-0-12-809194-4.
- Programming and Optimization for Intel® Architecture Hands-On Workshop (HOW) Series "Deep Dive" 2017. Session 1. Colfax International — colfaxresearch.com
- Slides from CS267 "Applications of Parallel Computers". Prof. Jim Demmel, EECS & Math Departments @ Berkeley (USA), 2015. demmel@berkeley.edu
- Lecture Slides from the Companion Site for the book Computer Architecture: A Quantitative Approach, Sixth Edition (from Ch. 01 to Ch. 07)

ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

3er curso

TEMA 1. Arquitectura de un multiprocesador- en-un-chip

Departamento de Ingeniería y Tecnología de Computadores
Universidad de Murcia